EIP-1559 가스 메커니즘 분석

개념적 배경

EIP-1559는 Ethereum의 가스 수수료 시장을 개선하기 위해 London 하드포크에서 도입된 메커니즘입니다. 기존의 단일 가스 가격 시스템을 Base Fee와 Priority Fee로 분리하여 수수료 예측 가능성을 높이고 네트워크 혼잡 상황을 개선했습니다.

EIP-1559가 해결하는 문제

1. **수수료 예측 불가능성**: 기존 경매 방식의 불확실성
2. **네트워크 혼잡**: 블록 크기 고정으로 인한 처리량 제한
3. **사용자 경험**: 복잡한 가스 가격 설정 과정
4. **경제적 비효율성**: 과도한 수수료 지불 문제

핵심 개념

- **Base Fee**: 네트워크 상태에 따라 자동 조정되는 기본 수수료
- **Priority Fee (Tip)**: 마이너에게 지불하는 우선순위 수수료
- **Max Fee**: 사용자가 지불할 의향이 있는 최대 수수료
- **Effective Gas Price**: 실제로 지불되는 가스 가격

구현 분석

1. Base Fee 계산 메커니즘

위치: `consensus/misc/eip1559/eip1559.go:56`

```go
func CalcBaseFee(config *params.ChainConfig, parent *types.Header) *big.Int {
// 첫 번째 EIP-1559 블록인 경우 초기 Base Fee 반환
if !config.IsLondon(parent.Number) {
return new(big.Int).SetUint64(params.InitialBaseFee)
}

parentGasTarget := parent.GasLimit / config.ElasticityMultiplier()
// 부모 블록의 가스 사용량이 목표와 같으면 Base Fee 유지
if parent.GasUsed == parentGasTarget {
return new(big.Int).Set(parent.BaseFee)
}

var (
num = new(big.Int)
denom = new(big.Int)
)

if parent.GasUsed > parentGasTarget {
// 목표보다 많이 사용된 경우 Base Fee 증가
num.SetUint64(parent.GasUsed - parentGasTarget)
num.Mul(num, parent.BaseFee)
num.Div(num, denom.SetUint64(parentGasTarget))
```

```
num.Div(num, denom.SetUint64(config.BaseFeeChangeDenominator()))

if  num.Cmp(common.Big1) < 0 {

return  num.Add(parent.BaseFee, common.Big1)

}

return  num.Add(parent.BaseFee, num)

} else {

// 목표보다 적게 사용된 경우 Base Fee 감소

num.SetUint64(parentGasTarget - parent.GasUsed)

num.Mul(num, parent.BaseFee)

num.Div(num, denom.SetUint64(parentGasTarget))

num.Div(num, denom.SetUint64(config.BaseFeeChangeDenominator()))


baseFee := num.Sub(parent.BaseFee, num)

if  baseFee.Cmp(common.Big0) < 0 {

baseFee = common.Big0

}

return  baseFee

}

}
```

Base Fee 계산 공식

증가 시 (GasUsed > GasTarget):

```
delta = (GasUsed - GasTarget) / GasTarget
```

```
baseFeeChange = max(1, ParentBaseFee * delta / BaseFeeChangeDenominator)

NewBaseFee = ParentBaseFee + baseFeeChange
```

감소 시 (GasUsed < GasTarget):

```
delta = (GasTarget - GasUsed) / GasTarget

baseFeeChange = ParentBaseFee * delta / BaseFeeChangeDenominator

NewBaseFee = max(0, ParentBaseFee - baseFeeChange)
```

주요 매개변수

| 매개변수 | 값 | 설명 |

|----------|-----|------|

| `InitialBaseFee` | 1 Gwei | 최초 Base Fee |

| `ElasticityMultiplier` | 2 | 블록 크기 탄력성 |

| `BaseFeeChangeDenominator` | 8 | Base Fee 변화율 제한 |

| `GasTarget` | GasLimit / 2 | 목표 가스 사용량 |

2. Effective Gas Price 계산

위치: `core/types/transaction.go:365`

```
func (tx *Transaction) calcEffectiveGasTip(dst *big.Int, baseFee *big.Int) error

if  baseFee == nil {

dst.Set(tx.inner.gasTipCap())

return  nil
```

```
}

var err error
gasFeeCap := tx.inner.gasFeeCap()
// Max Fee가 Base Fee보다 낮은 경우 에러
if gasFeeCap.Cmp(baseFee) < 0 {
err = ErrGasFeeCapTooLow
}

// Effective Tip = min(MaxPriorityFee, MaxFee - BaseFee)
dst.Sub(gasFeeCap, baseFee)
gasTipCap := tx.inner.gasTipCap()
if gasTipCap.Cmp(dst) < 0 {
dst.Set(gasTipCap)
}
return err
}
```

Effective Gas Price 공식

```
EffectiveGasPrice = BaseFee + min(MaxPriorityFee, MaxFee - BaseFee)
```

계산 과정:

1. MaxFee - BaseFee 계산

2. `MaxPriorityFee` 와 비교하여 작은 값 선택
3. `BaseFee` 에 선택된 값 추가

3. 가스 가격 오라클 (Gas Price Oracle)

위치: `eth/gasprice/gasprice.go:154`

3.1 Priority Fee 제안

```
func (oracle *Oracle) SuggestTipCap(ctx context.Context) (*big.Int, error) {
head, _ := oracle.backend.HeaderByNumber(ctx, rpc.LatestBlockNumber)
headHash := head.Hash()

// 캐시된 가격이 있으면 반환
oracle.cacheLock.RLock()
lastHead, lastPrice := oracle.lastHead, oracle.lastPrice
oracle.cacheLock.RUnlock()
if headHash == lastHead {
return new(big.Int).Set(lastPrice), nil
}

// 최근 블록들의 트랜잭션 분석
var results []*big.Int
for sent < oracle.checkBlocks && number > 0 {
go oracle.getBlockValues(ctx, number, sampleNumber,
```

```
oracle.ignorePrice, result, quit)

sent++

exp++

number--

}


// 결과 수집 및 백분위수 계산

if len(results) > 0 {

slices.SortFunc(results, func(a, b *big.Int) int {

return a.Cmp(b)

})

price = results[(len(results)-1)*oracle.percentile/100]

}


return new(big.Int).Set(price), nil

}
```

3.2 오라클 설정

```
type Config struct {

Blocks int // 분석할 블록 수 (기본값: 20)

Percentile int // 백분위수 (기본값: 60)

MaxHeaderHistory uint64 // 최대 헤더 히스토리

MaxBlockHistory uint64 // 최대 블록 히스토리
```

```
MaxPrice *big.Int // 최대 가격 제한
IgnorePrice *big.Int // 무시할 최소 가격
}
```

4. Fee History API

위치: `eth/gasprice/feehistory.go:85`

```
func (oracle *Oracle) processBlock(bf *blockFees, percentiles []float64) {
config := oracle.backend.ChainConfig()

// Base Fee 계산
if  bf.results.baseFee = bf.header.BaseFee;  bf.results.baseFee == nil {
bf.results.baseFee = new(big.Int)
}
// 다음 Base Fee 예측
if  config.IsLondon(big.NewInt(int64(bf.blockNumber + 1))) {
bf.results.nextBaseFee = eip1559.CalcBaseFee(config, bf.header)
}

// 가스 사용률 계산
bf.results.gasUsedRatio = float64(bf.header.GasUsed) /
float64(bf.header.GasLimit)
```

```
// 트랜잭션별 Effective Tip 분석
sorter := make([]txGasAndReward, len(bf.block.Transactions()))
for i, tx := range bf.block.Transactions() {
reward, _ := tx.EffectiveGasTip(bf.block.BaseFee())
sorter[i] = txGasAndReward{
gasUsed: bf.receipts[i].GasUsed,
reward: reward,
}
}
}
```

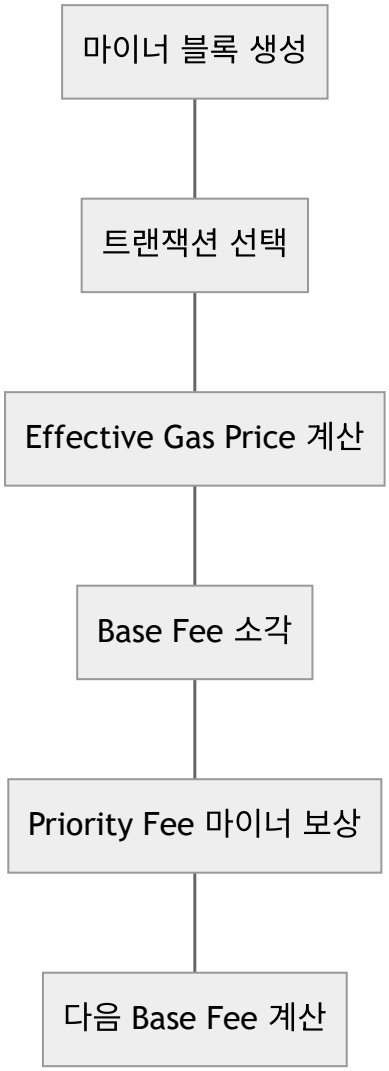
실행 흐름

1. 트랜잭션 생성 및 검증

```
graph TD
A[사용자 트랜잭션 생성] --> B[MaxFee, MaxPriorityFee 설정]
B --> C[트랜잭션 서명]
C --> D[풀 검증]
D --> E{MaxFee >= BaseFee?}
E -->|No| F[거부: ErrGasFeeCapTooLow]
E -->|Yes| G[Effective Tip 계산]
```

```
G -->  H[풀에 추가]
```

2. 블록 생성 시 수수료 처리



3. Base Fee 동적 조정

```
graph  TD

A[블록 완성]  -->  B{GasUsed vs GasTarget}

B -->|GasUsed > GasTarget|  C[Base Fee 증가]

B -->|GasUsed = GasTarget|  D[Base Fee 유지]

B -->|GasUsed < GasTarget|  E[Base Fee 감소]
```

```
C -->  F[다음 블록에 적용]

D -->  F

E -->  F
```

성능 최적화

1. 가스 가격 예측 최적화

```
// 캐싱을 통한 중복 계산 방지

oracle.cacheLock.RLock()

lastHead, lastPrice := oracle.lastHead, oracle.lastPrice

oracle.cacheLock.RUnlock()

if  headHash == lastHead {

return  new(big.Int).Set(lastPrice), nil

}
```

2. Effective Tip 계산 최적화

```
// 메모리 할당 최소화를 위한 in-place 계산

func (tx *Transaction) calcEffectiveGasTip(dst *big.Int, baseFee *big.Int) error

// 기존 big.Int 재사용으로 GC 압박 감소

dst.Sub(gasFeeCap, baseFee)

if  gasTipCap.Cmp(dst) < 0 {

dst.Set(gasTipCap)
```

```
}
return  err
}
```

3. 병렬 블록 분석

```
// 여러 블록을 병렬로 분석하여 응답 시간 단축
for  sent < oracle.checkBlocks && number > 0 {
go  oracle.getBlockValues(ctx, number, sampleNumber,
oracle.ignorePrice, result, quit)
sent++
number--
}
```

실제 사용 예시

1. EIP-1559 트랜잭션 생성

```
// 동적 수수료 트랜잭션 생성
tx := types.NewTx(&types.DynamicFeeTx{
ChainID: chainID,
Nonce: nonce,
To: &to,
```

```
Value: value,

Gas: gasLimit,

GasFeeCap: maxFee, // 최대 지불 의향 수수료

GasTipCap: maxTip, // 마이너 팁

Data: data,

})


// 서명

signedTx, err := types.SignTx(tx, signer, privateKey)
```

2. 가스 가격 추정

```
// Priority Fee 추정

tipCap, err := client.SuggestGasTipCap(ctx)

if  err != nil {

return  err

}


// Base Fee 조회 (최신 블록에서)

header, err := client.HeaderByNumber(ctx, nil)

if  err != nil {

return  err

}
```

```
baseFee := header.BaseFee


// Max Fee 계산 (Base Fee의 2배 + Tip으로 설정)
maxFee := new(big.Int).Add(
new(big.Int).Mul(baseFee, big.NewInt(2)),
tipCap,
)
```

3. Fee History 조회

```
// 최근 10블록의 수수료 히스토리 조회
feeHistory, err := client.FeeHistory(ctx, 10, nil, []float64{25, 50, 75})
if err != nil {
return err
}


// 백분위수별 Priority Fee 분석
for i, rewards := range feeHistory.Reward {
fmt.Printf("Block %d: 25th=%s, 50th=%s, 75th=%s\n",
feeHistory.OldestBlock.Uint64()+uint64(i),
rewards[0], rewards[1], rewards[2])
}
```

에러 처리

주요 에러 타입

| 에러 | 설명 | 처리 방법 |

|------|------|----------|

| `ErrGasFeeCapTooLow` | Max Fee < Base Fee | Max Fee 증가 |

| `ErrTipAboveFeeCap` | Tip > Max Fee | Tip 또는 Max Fee 조정 |

| `ErrTxGasPriceTooLow` | Tip < 최소 요구값 | Tip 증가 |

에러 처리 예시

```
func handleGasError(err error, tx *types.Transaction) error {
switch {
case errors.Is(err, core.ErrGasFeeCapTooLow):
// Base Fee보다 Max Fee가 낮음
return fmt.Errorf("increase maxFeePerGas above %s",
getCurrentBaseFee())
case errors.Is(err, core.ErrTipAboveFeeCap):
// Tip이 Max Fee보다 높음
return fmt.Errorf("maxPriorityFeePerGas (%s) > maxFeePerGas (%s)",
tx.GasTipCap(), tx.GasFeeCap())
default:
return err
}
```

```
}
```

관련 시스템

의존성 관계

- **블록체인 상태**: Base Fee 계산을 위한 이전 블록 정보
- **트랜잭션 풀**: 수수료 기반 우선순위 결정
- **마이닝**: 수익성 기반 트랜잭션 선택
- **RPC API**: 가스 가격 추정 서비스

상호작용 패턴

1. **Base Fee 전파**: 새 블록 생성 시 네트워크 전체에 Base Fee 업데이트
2. **Priority Fee 경쟁**: 사용자들 간의 우선순위 수수료 경쟁
3. **동적 조정**: 네트워크 혼잡도에 따른 자동 수수료 조정
4. **예측 서비스**: 오라클을 통한 적정 수수료 제안

영향 범위

- **사용자 경험**: 예측 가능한 수수료로 UX 개선
- **네트워크 효율성**: 탄력적 블록 크기로 처리량 향상
- **경제적 효과**: Base Fee 소각으로 ETH 디플레이션 압력
- **마이너 수익**: Priority Fee를 통한 수익 구조 변화

EIP-1559는 Ethereum의 수수료 시장을 근본적으로 개선하여 사용자 경험을 향상시키고 네트워크의 경제적 지속가능성을 높이는 핵심 메커니즘입니다.